UNIT II

Threads: Overview – Threading issues - CPU Scheduling – Basic Concepts – Scheduling Criteria – Scheduling Algorithms – Multiple-Processor Scheduling – Real Time Scheduling - The Critical- Section Problem – Synchronization Hardware – Semaphores – Classic problems of Synchronization – Critical regions – Monitors.

2 Marks

1. What is a thread? (APR'15, NOV '15)

A thread otherwise called a lightweight process (LWP) is a basic unit of CPU utilization, it comprises of a thread id, a program counter, a register set and a stack. It shares with other threads belonging to the same process its code section, data section, and operating system resources such as open files and signals.

2. What are the benefits of multithreaded programming? (NOV'14)

The benefits of multithreaded programming can be broken down into four major categories:

- Responsiveness
- Resource sharing
- Economy
- Utilization of multiprocessor architectures

3. Write the types of Thread?

- Kernel-supported threads (e.g. Mach and OS/2) - kernel of O/S sees threads and manages switching between threads i.e. in terms of analogy boss (OS) tells person (CPU) which thread in process to do next.
- User-level threads - supported above the kernel, via a set of library calls at the user level. Kernel only sees process as whole and is completely unaware of any threads i.e. in terms of analogy manual of procedures (user code) tells person (CPU) to stop current thread and start another (using library call to switch threads)

4. Compare user threads and kernel threads? (May'16)

User threads

User threads are supported above the kernel and are implemented by a thread library at the user level. Thread creation & scheduling are done in the user space, without kernel intervention. Therefore they are fast to create and manage blocking system call will cause the entire process to block.

Kernel threads

Kernel threads are supported directly by the operating system .Thread creation; scheduling and management are done by the operating system. Therefore they are slower to create & manage compared to user threads. If the thread performs a blocking system call, the kernel can schedule another thread in the application for execution

5. Define thread cancellation & target thread.

The thread cancellation is the task of terminating a thread before it has completed. A thread that is to be cancelled is often referred to as the target thread.

For example, if multiple threads are concurrently searching through a database and one thread returns the result, the remaining threads might be cancelled.

6. What are the different ways in which a thread can be cancelled?

Cancellation of a target thread may occur in two different scenarios:

- Asynchronous cancellation: One thread immediately terminates the target thread is called asynchronous cancellation.
- Deferred cancellation: The target thread can periodically check if it should terminate, allowing the target thread an opportunity to terminate itself in an orderly fashion.

7. Define CPU scheduling (APR'15)

CPU scheduling is the process of switching the CPU among various processes. CPU scheduling is the basis of multiprogrammed operating systems. By switching the CPU among processes, the operating system can make the computer more productive.

8. What is preemptive and non preemptive scheduling?

Under nonpreemptive scheduling once the CPU has been allocated to a process, the process keeps the CPU until it releases the CPU either by terminating or switching to the waiting state. Preemptive scheduling can preempt a process which is utilizing the CPU in between its execution and give the CPU to another process.

9. What is a Dispatcher? (NOV '11)(Apr'17)

The dispatcher is the module that gives control of the CPU to the process selected by the short-term scheduler. This function involves:

- Switching context

• Switching to user mode
• Jumping to the proper location in the user program to restart that program.

10. What is dispatch latency?

The time taken by the dispatcher to stop one process and start another running is known as dispatch latency.

11. What are the various scheduling criteria for CPU scheduling?

The various scheduling criteria are

• CPU utilization
• Throughput
• Turnaround time
• Waiting time
• Response time

12. Define throughput? (NOV '11)(ARPIL '14)

Throughput in CPU scheduling is the number of processes that are completed per unit time. For long processes, this rate may be one process per hour; for short transactions, throughput might be 10 processes per second.

13. What is turnaround time?

Turnaround time is the interval from the time of submission to the time of completion of a process. It is the sum of the periods spent waiting to get into memory, waiting in the ready queue, executing on the CPU, and doing I/O.

14. Define race condition? (ARPIL '14)

When several process access and manipulate same data concurrently, then the outcome of the execution depends on particular order in which the access takes place is called race condition. To avoid race condition, only one process at a time can manipulate the shared variable.

15. What is critical section problem? (APR '11) (NOV '18)

Consider a system consists of 'n' processes. Each process has segment of code called a critical section, in which the process may be changing common variables, updating a table, writing a file. When

one process is executing in its critical section, no other process can allowed to execute in its critical section.

16. What are the requirements that a solution to the critical section problem must satisfy? (NOV'14) (may'16)

The three requirements are

- Mutual exclusion
- Progress
- Bounded waiting

17. Define entry section and exit section.

The critical section problem is to design a protocol that the processes can use to cooperate. Each process must request permission to enter its critical section. The section of the code implementing this request is the entry section. The critical section is followed by an exit section. The remaining code is the remainder section

18. Give two hardware instructions and their definitions which can be used for implementing mutual exclusion.

- TestAndSet

```
boolean TestAndSet (boolean &target)
{
boolean rv = target;
target = true;
return rv;
}
```

- Swap

```
void Swap (boolean &a, boolean &b)
{
boolean temp = a;
a = b;
b = temp;
}
```

19. What is semaphores?

A semaphore 'S' is a synchronization tool which is an integer value that, apart from initialization, is accessed only through two standard atomic operations; wait and signal. Semaphores can be used to deal with the n-process critical section problem. It can be also used to solve various synchronization problems. The classic definition of 'wait'

```
wait (S)
{
while (S<=0)
;
S--;
}
```

The classic definition of 'signal'

```
signal (S)
{
S++;
}
```

20. Define busy waiting and spinlock?

When a process is in its critical section, any other process that tries to enter its critical section must loop continuously in the entry code. This is called as busy waiting and this type of semaphore is also called a spinlock, because the process while waiting for the lock.

21. What happens if the time allocated in a Round Robin Scheduling is very large? And what happens if the time allocated is very low?

It results in a FCFS scheduling. If time is too low, the processor through put is reduced. More time is spent on context switching

22. What is reader writers problem in shared file type of interprocess communication?

In this problem if reader is faster than writer or writer is faster than reader than problem is bound to creep in .if reader is faster than writer then it would try to read the memory location which is not written by writer process. if writer is faster than reader then it would just keep filling the buffers unboundedly. this problem is reader. Writer problem in shared file IPC. So there is need for synchronization between them.

23. Write down Scheduling Criteria?

- **CPU utilization** i.e. CPU usage - to maximize
- **Throughput** = number of processes that complete their execution per time unit - to maximize
- **Turnaround time** = amount of time to execute a particular process - to minimize
- **Waiting time** = amount of time a process has been waiting in the ready queue - to minimize
- **Response time** = amount of time it takes from when a job was submitted until it initiates its first response (output), **not** to time it completes output of its first response - to minimize

24. What is the difference between process and thread? (May 2017)

1. Threads are easier to create than processes since they don't require a separate address space.
2. Multithreading requires careful programming since threads share data structures that should only be modified by one thread at a time. Unlike threads, processes don't share the same address space.
3. Threads are considered lightweight because they use far less resources than processes.
4. Processes are independent of each other. Threads, since they share the same address space are interdependent, so caution must be taken so that different threads don't step on each other.
This is really another way of stating #2 above.
5. A process can consist of multiple threads.

25. What is Spooling?

Acronym for simultaneous peripheral operations on line. Spooling refers to putting jobs in a buffer, a special area in memory or on a disk where a device can access them when it is ready. Spooling is useful because device access data that different rates. The buffer provides a waiting station where data can rest while the slower device catches up the spooling.

26. List the advantage of Spooling?

1. The spooling operation uses a disk as a very large buffer.
2. Spooling is however capable of overlapping I/O operation for one job with processor operations for another job.

27. What is meant by CPU–I/O Burst Cycle, CPU burst, I/O burst?

- **CPU–I/O Burst Cycle** – Process execution consists of a *cycle* of CPU execution and I/O wait.

- **CPU burst** is length of time process needs to use CPU before it next makes a system call (normally request for I/O).
- **I/O burst** is the length of time process spends waiting for I/O to complete.

28. Define Aging and starvation? (APR' 14)

Starvation: Starvation is a resource management problem where a process does not get the resources it needs for a long time because the resources are being allocated to other processes.

Aging: Aging is a technique to avoid starvation in a scheduling system. It works by adding an aging factor to the priority of each request. The aging factor must increase the requests priority as time passes and must ensure that a request will eventually be the highest priority request (after it has waited long enough).

29. What is context switch? (APR '11)

In a multitasking operating system stops running one process and starts running another. Many operating systems implement concurrency by maintaining separate environments or "contexts" for each process. The amount of separation between processes, and the amount of information in a context, depends on the operating system but generally the OS should prevent processes interfering with each other, e.g. by modifying each other's memory.

A context switch can be as simple as changing the value of the program counter and stack pointer or it might involve resetting the MMU to make a different set of memory pages available.

30. What is a monitor? (NOV '15)

A **monitor** is a synchronization construct that allows threads to have both mutual exclusion and the ability to wait (block) for a certain condition to become true. **Monitors** also have a mechanism for signaling other threads that their condition has been met.

31. What are the uses of job queues, ready queue and device queue? (MAY'17)

The Operating System maintains the following important process scheduling queues −

- Job queue − This queue keeps all the processes in the system.
- Ready queue − This queue keeps a set of all processes residing in main memory, ready and waiting to execute. A new process is always put in this queue.
- Device queues − The processes which are blocked due to unavailability of an I/O device constitute this queue.

32. Define Medium Term Scheduler (NOV'16)

Medium-term scheduling is a part of **swapping**. It removes the processes from the memory. It reduces the degree of multiprogramming. The medium-term scheduler is in-charge of handling the swapped out-processes.

A running process may become suspended if it makes an I/O request. A suspended processes cannot make any progress towards completion. In this condition, to remove the process from memory and make space for other processes, the suspended process is moved to the secondary storage. This process is called **swapping**, and the process is said to be swapped out or rolled out. Swapping may be necessary to improve the process mix.

33. What is Critical Regions? List down various mechanisms used to deal with critical region problem.(NOV'16) (NOV '18)

- It is a code paths that access and manipulate shared data are called *critical regions*.
- It is a higher level construct that removes some of the programmer overhead

• They consist of two parts:

1. Variables that must be accessed under mutual exclusion.
2. A new language statement that identifies a critical region in which the variables are accessed.

- shared variable is used

```
shared T v;
example:
shared int v1;
 struct xyzzy
{
char * a;
 int b;
}
shared struct xyzzy v2;
```

- special language construct to access shared variable

```
 region v when B do S;
```

where B is a boolean condition and S is one or more statements

```
example: region v1 when (true) do v1++;
region v2 when (v2.b > 0)
 {
 printf("%s\n",v2.a);
```

b--;

}

The various mechanisms used to deal with critical region problems are :

1. Mutual exclusion
2. Semaphore

34. **Write about real time scheduling? (NOV 18)**

Real time system means that the system is subjected to real time, i.e., response should be guaranteed within a specified timing constraint or system should meet the specified deadline. For example: flight control system, real time monitors etc.

11 Marks

1. Write short notes on CPU scheduler (8) (NOV 13) (APR '11) (NOV '15)

CPU Scheduling:

Basic concepts:

DEFINITION:

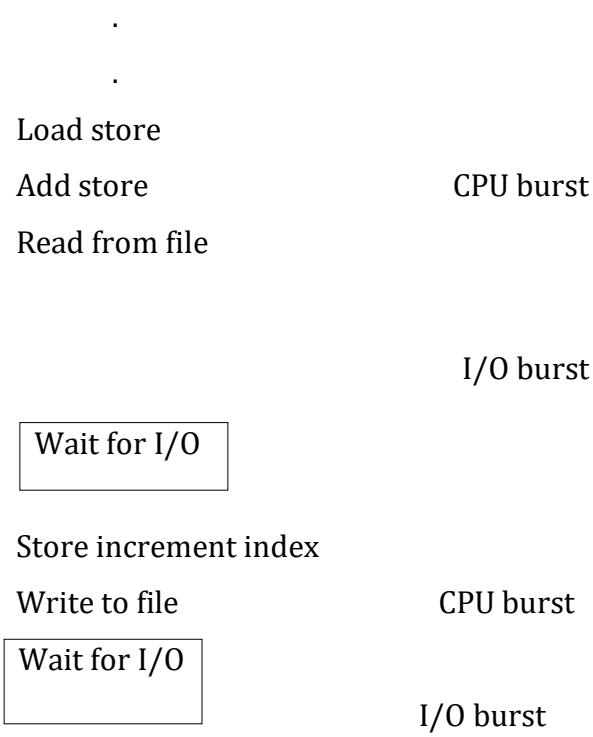
- CPU scheduling is the process of switching the CPU among various processes.
- CPU scheduling is the basic of multiprogrammed operating systems.
- By switching the CPU among processes, the operating system can make the computer more productive.

CPU – I/O BURST CYCLE:

The success of CPU scheduling depends on the property of processes:

- The process execution consists of a cycle of CPU execution and I/O wait.
- Processes alternate between these two states.
- Process execution begin with a CPU burst; followed by an I/O burst, then another CPU burst then another I/O burst and so on.
- Last CPU burst will end with a system request to terminate execution.
- An I/O bound program would have many, very short CPU bursts.
- A CPU bound program might have a few very long CPU bursts.

Fig: Alternating sequence of CPU and I/O bursts



Load store

Add store CPU burst

Read from file

Wait for I/O I/O burst

.

.

CPU Scheduler

- CPU scheduler selects one of the processes in the ready queue to be executed.
- There are two types of scheduling .they are
  1. Preemptive scheduling
  2. Non preemptive scheduling
- Preemptive scheduling-during process with the ,if is possible to remove the CPU from the process then it is called preemptive scheduling.
- Non preemptive scheduling-during processing with the CPU from the process then it is not possible to remove the CPU from the process then it is called non-preemptive scheduling.
- CPU scheduling decisions may take place when a process:

1. Switches from running to waiting state.
2. Switches from running to ready state.
3. Switches from waiting to ready.
4. Terminates.

- Scheduling under 1 and 4 is non-preemptive.
- All other scheduling is *preemptive.*

Dispatcher

- Dispatcher module gives control of the CPU to the process selected by the short-term scheduler; this involves:
  - switching context
  - switching to user mode
  - jumping to the proper location in the user program to restart that program
- *Dispatch latency* – time it takes for the dispatcher to stop one process and start another running.

Scheduling Criteria

- CPU utilization – keep the CPU as busy as possible
- Throughput – # of processes that complete their execution per time unit
- Turnaround time – amount of time to execute a particular process
- Waiting time – amount of time a process has been waiting in the ready queue

- Response time – amount of time it takes from when a request was submitted until the first response is produced, not output (for time-sharing environment)

Optimization Criteria

- Max CPU utilization
- Max throughput
- Min turnaround time
- Min waiting time
- Min response time

Scheduling algorithms:

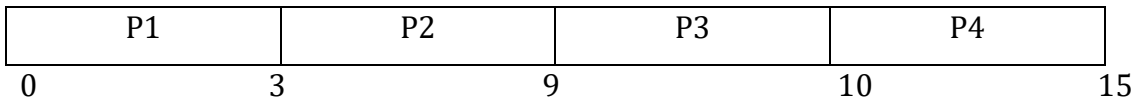
- First –come first served scheduling(FCFS)
- Shortest-job-first scheduling(SJF)
- Priority scheduling
- Round-robin scheduling(RR)
- Multilevel queue scheduling
- Multilevel feedback queue scheduling

2. Explain FCFS (6) (NOV 13) (NOV 18)

- It is a non pre emptive algorithm
- The process which requests the CPU first us allocated to the CPU first.
- Demerit –A long CPU bound job may take the CPU and force short job to wait for a long tern called convoy effect
- Gantt chart -It represents the order in which the process is executed
- Example

Process	Burst time
P1	3
P2	6
P3	4
P4	2

Gantt chart:



Waiting time

Process	waiting time
P1	0
P2	3
P3	9
P4	13

Average waiting time=(0+3+9+13)/4 = 6.25 ms

Turn around time (TAT):

TAT=waiting time + burst time

Process	TAT
P1	3
P2	9
P3	13
P4	15

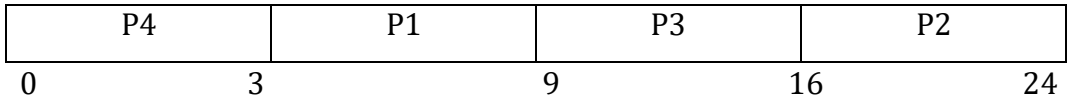
Average TAT=(3+9+13+15)/4=10 ms

3.Write short notes on shortest job scheduling (6)

- CPU is allocated to a job with smaller CPU burst that is the shortest CPU burst job will have the higher priority over other jobs
- Two schemes:
  - nonpreemptive – once CPU given to the process it cannot be preempted until completes its CPU burst.
  - preemptive – if a new process arrives with CPU burst length less than remaining time of current executing process, preempt. This scheme is known as the Shortest-Remaining-Time-First (SRTF).
- SJF is optimal – gives minimum average waiting time for a given set of processes.
- Example

Process	Burst time
P1	6
P2	8
P3	7
P4	3

Gantt chart:



Waiting time

Process	waiting time
P1	3
P2	16
P3	9
P4	0

Average waiting time=(3+16+9+0)/4 = 7 ms

Turn around time (TAT):

TAT=waiting time + burst time

Process	TAT
P1	9
P2	24
P3	16
P4	3

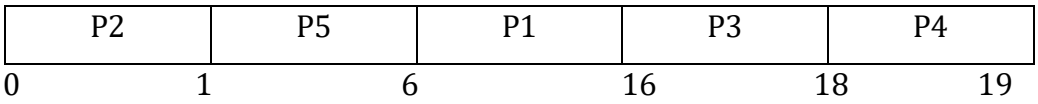
Average TAT=(9+24+16+3)/4=13 ms

4.Write short notes on priority scheduling (6)

- A priority number (integer) is associated with each process
- The CPU is allocated to the process with the highest priority (smallest integer ≡ highest priority).
  - Preemptive
  - nonpreemptive
- preemptive-preempt the CPU if the priority of the newly arrived process is higher than the priority of the currently running process
- non preemptive - allows the currently running process to complete its CPU burst.
- SJF is a priority scheduling where priority is the predicted next CPU burst time.
- Problem in priority scheduling is Starvation –i.e low priority processes may never execute.
- Solution -Aging – as time progresses increase the priority of the process.
- Example

Process	Burst time	Priority
P1	10	3
P2	1	1
P3	2	4
P4	1	5
P5	5	2

Gantt chart:



Waiting time

Process	waiting time
P1	6
P2	0
P3	16
P4	18
P5	1

Average waiting time=(6+0+16+18+1)/5 = 8.2ms

Turn around time (TAT):

TAT=waiting time + burst time

Process	TAT
P1	16
P2	1
P3	18
P4	19
P5	6

Average TAT=(16+1+18+19+6)/5=12 ms

5. Explain Round Robin (RR) (NOV '15) (6)

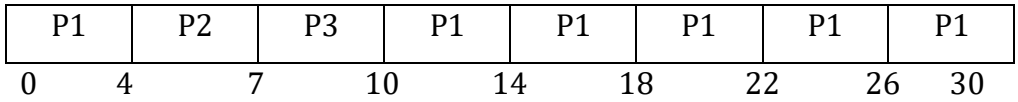
- It is a pre-emptive scheduling

- Each process gets a small unit of CPU time (*time quantum*), usually 10-100 milliseconds. After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are *n* processes in the ready queue and the time quantum is *q*, then each process gets 1/*n* of the CPU time in chunks of at most *q* time units at once. No process waits more than (*n*-1)*q* time units.
- Performance
  - *q* large ⇒ FIFO
  - *q* small ⇒ *q* must be large with respect to context switch, otherwise overhead is too high.

☞Example given time quantum=4ms

Process	Burst time
P1	24
P2	3
P3	3

Gantt chart:



Waiting time

Process	waiting time
P1	10-4=6
P2	4
P3	7

Average waiting time=(6+4+7)/3 = 5.6 ms

Turn around time (TAT):

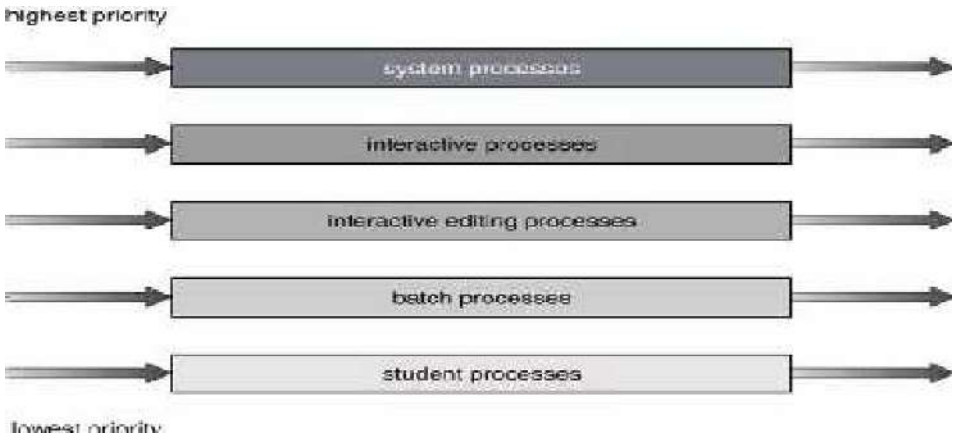
TAT=waiting time + burst time

Process	TAT
P1	30
P2	7
P3	10

Average TAT=(30+7+10)/3=15.6 ms

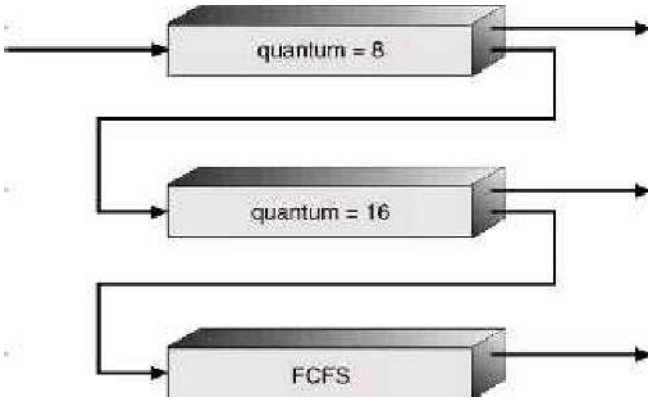
6.Explain Multi level queue scheduling (6)(NOV '11)

- Ready queue is partitioned into separate queues:
  foreground (interactive)
  background (batch)
- Each process is permanently assigned to one queue based on some property eg: process type
- Each queue has its own scheduling algorithm,
  foreground – RR
  background – FCFS
- Scheduling must be done between the queues.
  - Fixed priority scheduling; (i.e., serve all from foreground then from background). Possibility of starvation.
  - Time slice – each queue gets a certain amount of CPU time which it can schedule amongst its processes; i.e., 80% to foreground in RR
  - 20% to background in FCFS



7.Explain Multilevel Feedback Queue (7)

- A process can move between the various queues; aging can be implemented this way.
- Multilevel-feedback-queue scheduler defined by the following parameters:
  - number of queues
  - scheduling algorithms for each queue
  - method used to determine when to upgrade a process
  - method used to determine when to demote a process
  - method used to determine which queue a process will enter when that process needs service



Example of Multilevel Feedback Queue

- Three queues:
  - Q_0 – time quantum 8 milliseconds
  - Q_1 – time quantum 16 milliseconds
  - Q_2 – FCFS
- Scheduling
  - A new job enters queue Q_0 which is served FCFS. When it gains CPU, job receives 8 milliseconds. If it does not finish in 8 milliseconds, job is moved to queue Q_1.
  - At Q_1 job is again served FCFS and receives 16 additional milliseconds. If it still does not complete, it is preempted and moved to queue Q_2.

8. Write short notes on Multiple-Processor Scheduling (4)(NOV '11)(APR'12)

- Here each queue can have separate processor
- Suppose if any of the queue is empty then that processor connected to it become idle. To avoid this problem we have to use common ready queue
- All the processes are sent to common ready queue and the processor has to take the process from that queue therefore here the processor is self scheduling
- Here also problem arise if two processor selects same process to avoid this one of the processor has to act as master for other processors that is master slave relationship
- The master has to select job from common ready queue and allocate that process to any one of slave processor.

Real-Time Scheduling

- *Hard real-time* systems – required to complete a critical task within a guaranteed amount of time.
- *Soft real-time* computing – requires that critical processes receive priority over less fortunate ones.

9. Write in detail about Critical section problem OR what is critical section problem and explain two process solution and multiple process solutions? (11) (NOV 11) (APR'15)

The Critical-Section Problem

- *n* processes all competing to use some shared data
- Each process has a code segment, called *critical section*, in which the shared data is accessed.
- Problem – ensure that when one process is executing in its critical section, no other process is allowed to execute in its critical section.
  - The critical section problem is to design a protocol that the processes can use to co-operate.
  - Each process must request permission to enter its critical section.
  - The section of code implementing this request is the entry section,
  - The critical section is followed by an exit section.
  - The remaining code is the remainder section.

GENERAL STRUCTURE OF TYPICAL PROCESS P1,

do

{

Entry section

Critical section

Exit section

Remainder section

} while (TRUE);

Solution to Critical-Section Problem

- Mutual Exclusion. If process P_i is executing in its critical section, then no other processes can be executing in their critical sections.
- Progress. If no process is executing in its critical section and there exist some processes that wish to enter their critical section, then the selection of the processes that will enter the critical section next cannot be postponed indefinitely.
- Bounded Waiting. A bound must exist on the number of times that other processes are allowed to enter their critical sections after a process has made a request to enter its critical section and before that request is granted.Assume that each process executes at a nonzero speed .No assumption concerning relative speed of the *n* processes.

TWO PROCESS SOLUTION:

Algorithm 1

- Process p0 and p1 both are cooperating through a shared variable turn.
- Turn may be 0 or 1 , if it is 0 p0 will be executed that is p0 ‘s critical section executes if it is 1,p1 will be executes that is p1 ‘s critical section executes.
- Shared variables:
  - int turn;
    initially turn = 0
  - turn = i $\Rightarrow$ P_i can enter its critical section
- Process P_i

```
do {
        while (turn != i) ;
                critical section
        turn = j;
                remainder section
} while (1);
```

- Satisfies mutual exclusion, but not progress
  (because strict alternation is enforced)

Algorithm 2

- Shared variables flag[0] and flag[1] are used to synchronize two computing processes.
- flag[0] and flag[1] are initially set to false.
- Whenever a process p0 intends to enter its CS ,it indicates by setting flag[0] which is accessible by other coo.perating process p1.now p0 checks whether flag[1] is set,if not then p0 enters its CS.
- Shared variables
  - boolean flag[2];
    initially flag [0] = flag [1] = false.
  - flag [i] = true $\Rightarrow$ P_i ready to enter its critical section

Process P_i

```
do {
flag[i] := true;
        while (flag[j]) ;
```

critical section

flag [i] = false;

remainder section

} while (1);

- Satisfies mutual exclusion, but not progress requirement

(flag[0]:=true; flag[1]:=true; ➔ both P_i looping in while())

Algorithm 3

- If flag[0] is set and turn equals to zero then p0 enters CS.
- If flag[1] ==1 and turn==1 then p1 enters to CS.
- Combined shared variables of algorithms 1 and 2.
- Process P_i

do {

flag [i]:= true;

turn = j;

while (flag [j] and turn = j) ;

critical section

flag [i] = false;

remainder section

} while (1);

- Meets all three requirements (mutual exclusion, progress, bounded waiting); solves the critical-section problem for two processes.

10. Explain Bakery Algorithm OR Multiple process solution (5) (APR'15)

- Before entering its critical section, process receives a number. Holder of the smallest number enters the critical section.
  - Holder of the smallest number enters the critical section.
- If processes P_i and P_j receive the same number, if $i < j$, then P_i is served first; else P_j is served first.
- Shared data

boolean choosing[n];

int number[n];

Data structures are initialized to false and 0 respectively

Algorithm

do {

```
choosing[i] = true;
number[i] = max(number[0], number[1], …, number [n – 1])+1;
choosing[i] = false;
for (j = 0; j < n; j++) {
        while (choosing[j]) ;
        while ((number[j] != 0) && (number[j], j) < (number[i],i)) ;
}
    critical section
number[i] = 0;
    remainder section
} while (1);
```

11. Write short notes on Synchronization Hardware (5) (NOV 11)(NOV 13)

- Test and modify the content of a word atomically.

```
boolean TestAndSet(boolean &target) {
        boolean rv = target;
        target = true;

        return rv;
    }
```

Mutual Exclusion with Test-and-Set

- Shared data:

```
boolean lock = false;
```

- Process P_i

```
do {
while (TestAndSet(lock)) ;
critical section
lock = false;
remainder section        }
while(1);
```

Synchronization Hardware (Swap)

- Atomically swap two variables.

```
void Swap(boolean &a, boolean &b) {
```

```
        boolean temp = a;
        a = b;
        b = temp;
    }
```

Mutual Exclusion with Swap

- Shared data (initialized to false):

```
boolean lock;
```

- Local data:

```
boolean key;
```

- Process P_i

```
do {
        key = true;
        while (key == true)
                Swap(lock, key);
            critical section
        lock = false;
            remainder section
} while(1);
```

Bounded Mutual Exclusion with T&S

- Shared data (initialized to false):

```
boolean lock;
        boolean waiting[n];
```

- Process P_i

```
do {
waiting[i] = true;
key = true;
while (waiting[i] && key) key = TestAndSet(lock);
waiting[i] = false;
critical section
j=(i+1) % n;
while ((j!=i) && !waiting[j])
```

```
                j = (j+1) % n;
            if (j == i)     lock = false;
            else    waiting[j] = false;
                    remainder section
        } while(1);
```

12.Write short notes on Semaphores(NOV '12, NOV '15) (NOV 18)

- Synchronization tool that does not require busy waiting.
- A Semaphore *S* is an integer variable can only be accessed via two indivisible (atomic) operations wait and signal.

```
wait (S):
        while S≤ 0 do no-op;
        S--;

signal (S):
        S++;
```

  - There are 2 processes p1 and p2 .p1 checks the valur of S,initially S=1 therefore s!=0. Since S- - makes s=0 so p1 enter to its CS .At the same time p2 also want to execute CS ,it checks S value which is already 0 so that it understands some process is in CS.so that p2 has to wait.
  - After p1 finishes,it increments S by 1 therefore now S=1.now p2 checks S value,which is not equal to 0 ,so that p2 will decrement S by 1 therefore S=0 no p2 enter to its CS.

Critical Section of *n* Processes

- N processes share a semaphore mutex initialized to 1.
- Shared data:

```
semaphore mutex;    // initially mutex = 1
```

- Process P_i:

```
do {
  wait(mutex);
    critical section
     signal(mutex);
    remainder section
```

```
} while (1);
```

Semaphore Implementation

- Define a semaphore as a record

```
typedef struct {
  int value;
struct process *L;
} semaphore;
```

- Assume two simple operations:
  - block() suspends the process that invokes it.
  - wakeup(*P*) resumes the execution of a blocked process P.

Implementation

- Semaphore operations now defined as

```
wait(S):
S.value--;
if (S.value < 0)
{
add this process to S.L;
block();
}
signal(S):
S.value++;
if (S.value <= 0) {
remove a process P from S.L;
wakeup(P);
}
```

Semaphore as a General Synchronization Tool

- Execute *B* in P_j only after *A* has executed in P_i
- Use semaphore *flag* initialized to 0
- Code:

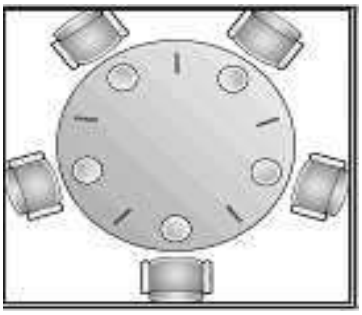
P_i P_j

⋮ ⋮

A *wait*(*flag*)

signal(*flag*) B

13.Write short notes on Dining-Philosophers Problem (6) (NOV'14) (NOV 18)



- Five philosophers are seated in a circular table
- A philosopher needs two forks to eat.
- When a philosopher gets hungry, she tries to pick up the left and right chopsticks.
  - ✓ A philosopher may pick up only one chopstick at a time.
- When a philosopher finished eating she puts down both chopsticks.
- The life of a philosopher consists of alternate periods of eating and thinking.
- Philosopher *i*:

```
do {
        wait(chopstick[i])
        wait(chopstick[(i+1) % 5])
                …
                eat
                …
        signal(chopstick[i]);
        signal(chopstick[(i+1) % 5]);
                …
                think
                …
        } while (1);
```

- The solution guarantees no two neighbors are eating simultaneously but may create a deadlock.
- Possible remedies:

1. Allow at most 4 philosophers to be sitting at the table.

2. Allow a philosopher to pickup his chopsticks only if both chopsticks are available.
3. An odd philosopher picks up first her left chopstick and then her right chopstick, an even philosopher picks up first her right chopstick and then her left chopstick.

- A deadlock free solution may not be starvation free.

14. Write short notes on Readers-Writers Problem (6)(APR '14,NOV'14)

- A data object is shared among several processes.
- Readers-processes that only want to read the shared objects.
- Writers-processes that want to update that is read and write the shared data object.
- More than one readers are allowed to access the shared object simultaneously.
- Writers must have exclusive access to the shared object.
- No reader will be kept waiting unless a writer has already obtained permissions to use the shared object
  - ✓ Writers may starve.
- Shared data

  semaphore mutex, wrt;

  Initially

  mutex = 1, wrt = 1, readcount = 0

  Readers-Writers Problem Writer Process

  wait(wrt);

  …

  writing is performed

  …

  signal(wrt);

  Readers-Writers Problem Reader Process

  wait(mutex);

  readcount++;

  if (readcount == 1)

  wait(wrt);

  signal(mutex);

  …

  reading is performed

…

wait(mutex);

readcount--;

if (readcount == 0)

signal(wrt);

signal(mutex):

- The first reader to get access to database. subsequent readers increment a counter read count. As reader leaves ,they decrement the counter and the last one does an signal to the blocked writer.

15.Explain producer-consumer problem (7)(APR '14)

- A common paradigm for cooperating processes, A producer process produces information that is consumed by a consumer process.
- Have a buffer of items filled by the producer and emptied by the consumer
  - unbounded-buffer no limit on the size of the buffer.
  - bounded-buffer assumes that there is a fixed buffer size.
- Buffer is a shared memory .Producer and consumer run concurrently and must be synchronized. In bounded buffer, the consumer must wait if the buffer is empty and the producer must wait if the buffer is full.

In Unbounded Buffer,the consumer may have to wait for new items,but the producer can always produce new items

Bounded-Buffer Problem

- Shared data

  semaphore full, empty, mutex;

  Initially:

  full = 0, empty = n, mutex = 1

  Bounded-Buffer Problem Producer Process

```
do {
        …
    produce an item in nextp
        …
    wait(empty);
    wait(mutex);
```

```
                    …
                add nextp to buffer
                    …
                signal(mutex);
                signal(full);
        } while (1);
```

- Count =number of buffers
- The producer produces an item and access buffer by decrementing count by 1 and sets mutex=0 and then adds item to buffer. after that producer leaves CS by setting mutex=1 ,increments full buffers by 1.

Bounded-Buffer Problem Consumer Process

```
        do {
                wait(full)
                wait(mutex);
                    …
                remove an item from buffer to nextc
                    …
                signal(mutex);
                signal(empty);
                    …
                consume the item in nextc
                    …
        } while (1);
```

- The consumer decrementing full buffer by 1 and then enters CS by setting mutex=0 after that consumes an item ,while leaving CS sets mutex=1 and increments empty buffers by 1.

16.Write short notes on Critical region (8)

- High-level synchronization construct
- A shared variable *v* of type *T*, is declared as:

```
        v: shared T
```

- Variable *v* accessed only inside statement

region v when B do S

where *B* is a boolean expression.

While statement *S* is being executed, no other process can access variable *v*.

- Regions referring to the same shared variable exclude each other in time.
- When a process tries to execute the region statement, the Boolean expression *B* is evaluated. If *B* is true, statement *S* is executed. If it is false, the process is delayed until *B* becomes true and no other process is in the region associated with *v*.

Example – Bounded Buffer

- Shared data:

```
struct buffer {
        int pool[n];
        int count, in, out;
}
```

Bounded Buffer Producer Process

- Producer process inserts nextp into the shared buffer

```
region buffer when( count < n) {
pool[in] = nextp;
in:= (in+1) % n;
count++;
}
```

Bounded Buffer Consumer Process

- Consumer process removes an item from the shared buffer and puts it in nextc

```
region buffer when (count > 0) {
        nextc = pool[out];
                out = (out+1) % n;
                count--;
        }
```

Implementation region *x* when *B* do *S*

- Associate with the shared variable *x*, the following variables:

semaphore mutex, first-delay, second-delay;

int first-count, second-count;

- Mutually exclusive access to the critical section is provided by mutex.

- If a process cannot enter the critical section because the Boolean expression B is false, it initially waits on the first-delay semaphore; moved to the second-delay semaphore before it is allowed to reevaluate *B*.
- Keep track of the number of processes waiting on first-delay and second-delay, with first-count and second-count respectively.

- The algorithm assumes a FIFO ordering in the queuing of processes for a semaphore.

- For an arbitrary queuing discipline, a more complicated implementation is required.

17.Write short notes on Monitors (OR) Explain the structure of monitors . Give the solution for Dining-philosopher problem using monitor (11) (NOV'16)

- High-level synchronization construct that allows the safe sharing of an abstract data type among concurrent processes.

```
monitor monitor-name
        {
shared variable declarations
        procedure body P1 (...) {
                . . .
        }
procedure body P2 (...) {
                . . .
        }
                procedure body Pn (...)
        {               . . .
        }
        {
                initialization code
        }
```

}

- To allow a process to wait within the monitor, a condition variable must be declared, as

  condition x, y;

- Condition variable can only be used with the operations wait and signal.
  - The operation

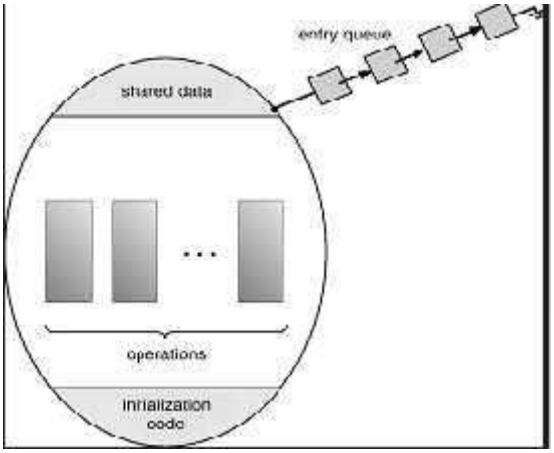
    x.wait();

means that the process invoking this operation is suspended until another process invokes

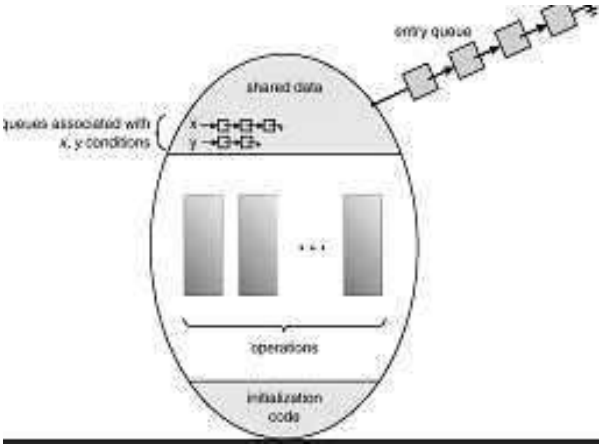
    x.signal();

  - The x.signal operation resumes exactly one suspended process. If no process is suspended, then the signal operation has no effect.

Schematic view of a monitor:



Monitor with condition variables



Dining Philosophers Example

monitor dp

enum {thinking, hungry, eating} state[5];

condition self[5];

void pickup(int i) // following slides

```
void putdown(int i) // following slides
void test(int i)                    // following slides
void init() {
        for (int i = 0; i < 5; i++)
state[i] = thinking;
        }
        }
        void pickup(int i) {
        state[i] = hungry;
                test[i];
                if (state[i] != eating)
                        self[i].wait();
        }
        void putdown(int i) {
                state[i] = thinking;
                // test left and right neighbors
                test((i+4) % 5);
                test((i+1) % 5);
        }
        void test(int i) {
                if ( (state[(i + 4) % 5] != eating) &&
                  (state[i] == hungry) &&
                  (state[(i + 1) % 5] != eating)) {
                        state[i] = eating;
                        self[i].signal();
                }
        }
```

Monitor Implementation Using Semaphores

- Variables

```
{
                semaphore mutex; // (initially = 1)
                semaphore next;     // (initially = 0)
```

```
int next-count = 0;
```

- Each external procedure *F* will be replaced by

```
wait(mutex);
   …
  body of F;
   …
if (next-count > 0)
        signal(next);
else
        signal(mutex);
```

- Mutual exclusion within a monitor is ensured.

Monitor Implementation

- For each condition variable *x*, we have:

```
semaphore x-sem; // (initially = 0)
int x-count = 0;
```

- The operation x.wait can be implemented as:

```
x-count++;
if (next-count > 0)
        signal(next);
else
        signal(mutex);
wait(x-sem);
x-count--;
```

- The operation x.signal can be implemented as:

```
if (x-count > 0) {
        next-count++;
        signal(x-sem);
        wait(next);
        next-count--;
}
```

- *Conditional-wait* construct: x.wait(c);

- c – integer expression evaluated when the wait operation is executed.
- value of c (a *priority number*) stored with the name of the process that is suspended.
- when x.signal is executed, process with smallest associated priority number is resumed next.

- Check two conditions to establish correctness of system:
  - User processes must always make their calls on the monitor in a correct sequence.
  - Must ensure that an uncooperative process does not ignore the mutual-exclusion gateway provided by the monitor, and try to access the shared resource directly, without using the access protocols.

18. Explain real time scheduling?

Real-Time Scheduling

Real-time computing is divided into two types:

- HARD REAL-TIME SYSTEMS
- SOFT REAL-TIME COMPUTING

HARD REAL-TIME SYSTEMS:

- **Hard real-time** systems are required to complete a critical task within a guaranteed amount of time.
- Generally, a process is submitted along with a statement of the amount of time in which it needs to complete or perform I/O.
- The scheduler then either admits the process, guaranteeing that the process will complete on time, or rejects the request as impossible.
- This is known as **resource reservation.**
- Such a guarantee requires that the scheduler know exactly how long each type of operating-system function takes to perform, and therefore each operation must be guaranteed to take a maximum amount of time.
- Such a guarantee is impossible in a system with secondary storage or virtual memory, as we shall show in the next few chapters, because these subsystems cause unavoidable and unforeseeable variation in the amount of time to execute a particular process.
- Therefore, hard real-time systems are composed of special-purpose software running on hardware dedicated to their critical process, and lack the full functionality of modern computers and operating systems.

SOFT REAL TIME COMPUTING:

- **Soft real-time** computing is less restrictive. It requires that critical processes receive priority over less fortunate ones.

- Although adding soft real-time functionality to a time-sharing system may cause an unfair allocation of resources and may result in longer delays, or even starvation, for some processes, it is at least possible to achieve.
- The result is a general-purpose system that can also support multimedia, high-speed interactive graphics, and a variety of tasks that would not function acceptably in an environment that does not support soft real-time computing. Implementing soft real-time functionality requires careful design of the scheduler and related aspects of the operating system.
- First, the system must have priority scheduling, and real-time processes must have the highest priority.
- The priority of real-time processes must not degrade over time, even though the priority of non-real-time processes may. Second, the dispatch latency must be small. The smaller the latency, the faster a real-time process can start executing once it is runnable. The high-priority process would be waiting for a lower-priority one to finish. This situation is known as **priority inversion.**
- In fact, a chain of processes could all be accessing resources that the high-priority process needs. This problem can be solved via the **priority-inheritance protocol,** in which all these processes (the ones accessing resources that the high-priority process needs) inherit the high priority until they are done with the resource in question. When they are finished, their priority reverts to its original value.

The **conflict phase** of dispatch latency has two components:

1. Preemption of any process running in the kernel
2. Release by low-priority processes resources needed by the high-priority process

As an example, in Solaris 2, the dispatch latency with preemption disabled is over 100 milliseconds. However, the dispatch latency with preemption enabled is usually reduced to 2 milliseconds.

THREADS: OVERVIEW

DEFINITION

- A thread called as a light weight process (LWP) is a basic unit of CPU utilization.
- It comprises a thread ID, a program counter, a register set and a stack.
- It shares with other threads belonging to the same process its code section, data section, and other operating system resources such as open files and signals.

Benefits of multi threaded programming

1. Responsiveness
2. Resource sharing
3. Economy

4. Utilization of multiprocessor architecture.

User and Kernel Threads

Threads maybe provided at either the user level, for user threads or by the kernel for kernel threads.

USER THREADS:

- Supported above the kernel and are implemented by a thread library at the user level.
- Library provides support for thread execution, scheduling and management with no support from kernel.
- User threads are fast to create and manage.
- Blocking system call will cause the entire process to block.

KERNEL THREADS:

- Supported directly by the operating system.
- Kernel performs thread creation, scheduling and management in kernel space.
- They are slower to create and manage than the user thread.
- If thread performs blocking system call, the kernel can schedule another thread in the application for execution.

19. Explain in detail about the threading issues (APR'15)

Threading Issues:

FORK AND EXEC SYSTEM CALLS:

Fork system call – create a separate, duplicate process.

Exec system call – runs an executable file.

In multithreaded program ; semantics of fork and exec change

Fork

- duplicate all threads

- duplicates only the thread that invoked fork()

Exec

- Program specified in the parameter to exec will replace the entire process.

Thread cancellation

It is a task of terminating a thread before it has completed.

Example:

When a user presses a button on a web browser that stops a web page from loading any further. Often a web is loaded in a separate thread when a user pressed the stop button, the thread loading the page is cancelled.

Target thread:

A thread that is to be cancelled is often referred to as the target thread.

Cancellations of a target thread may occur in two situations:

Two general approaches:

1. **Asynchronous cancellation** terminates the target thread immediately
2. **Deferred cancellation** allows the target thread to periodically check if it should be cancelled
   - ✓ Allow cancellation at safe points
   - ✓ Pthreads refer to safe point as cancellation points

Thread pools

Motivating example:

A web server creates anew thread to service each request.

Two concerns:

1. The amount of time required to create the thread prior to servicing the request ,compounded with the fact that this thread will be discarded once it has completed its work that is overhead to create thread

2. No limit on the number of thread created, may exhaust system resources, such as CPU time or memory .

To overcome the above said problem we need thread pools.

General idea:

- ➢ Create a pool of threads at process startup.
- ➢ If request comes in, then wakeup a thread from pool, assign the request to it,if no thread available ,server waits until one is free
- ➢ After completing the service, thread returns to pool.

Advantages:

- ✓ Usually slightly faster to service a request with an existing thread than create a new thread
- ✓ Allows the number of threads in the application(s) to be bound to the size of the pool

20. Describe the difference between wait (A) where A is a semaphore and B Wait() ,where B is a condition variable in a monitor. (NOV'14)

Monitor

- ➢ A monitor is a **set of multiple** routines which are protected by a mutual exclusion lock.
- ➢ None of the routines in the monitor can be executed by a thread until that thread acquires the lock.

- This means that only ONE thread can execute within the monitor at a time.
- Any other threads must wait for the thread that's currently executing to give up control of the lock.

However, a thread can actually suspend itself inside a monitor and then wait for an event to occur. If this happens, then another thread is given the opportunity to enter the monitor. The thread that was suspended will eventually be notified that the event it was waiting for has now occurred, which means it can wake up and reacquire the lock.

Semaphore

A semaphore is a simpler construct than a monitor because it's just a lock that protects a shared resource – and not a set of routines like a monitor. The application must acquire the lock before using that shared resource protected by a semaphore.

Example of a Semaphore – a Mutex

A mutex is the most basic type of semaphore, and mutex is short for mutual exclusion. In a mutex, only one thread can use the shared resource at a time. If another thread wants to use the shared resource, it must wait for the owning thread to release the lock.

Differences between Monitors and Semaphores

Both Monitors and Semaphores are used for the same purpose – thread synchronization. But, monitors are simpler to use than semaphores because they handle all of the details of lock acquisition and release. An application using semaphores has to release any locks a thread has acquired when the application terminates – this must be done by the application itself. If the application does not do this, then any other thread that needs the shared resource will not be able to proceed.

Another difference when using semaphores is that every routine accessing a shared resource has to explicitly acquire a a lock before using the resource. This can be easily forgotten when coding the routines dealing with multithreading . Monitors, unlike semaphores, automatically acquire the necessary locks.

Condition variable in Monitor:

A **condition variable** is basically a container of threads that are waiting on a certain **condition**. **Monitors** provide a mechanism for threads to temporarily give up exclusive access in order to wait for some **condition** to be met, before regaining exclusive access and resuming their task.

21. (a)List the requirements that must be satisfied by critical section problem (4 marks)

(b) Discuss the relationship between user threads and kernel threads (5 marks)

(c) List the names of thread libraries (2 marks) (APR'17)

(a) A solution to critical section problem must satisfy the following requirements:

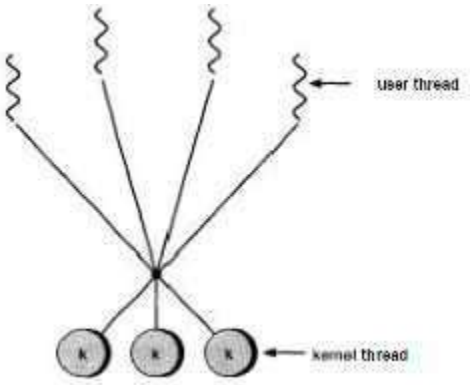
- **Mutual exclusion:** When a thread is executing in its critical section, no other threads can be executing in their critical sections.
- **Progress:** If no thread is executing in its critical section, and if there are some threads that wish to enter their critical sections, then one of these threads will get into the critical section.
- **Bounded waiting:** After a thread makes a request to enter its critical section, there is a bound on the number of times that other threads are allowed to enter their critical sections, before the request is granted.

(b) There must exist a relationship between user threads and kernel threads. The three common ways of establishing this relationship are as follow:

- One-to-One Model
- One-to-Many Model
- Many-to-One Model
- Many-to-Many Model

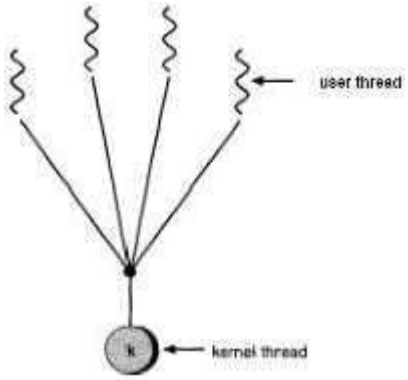
Many to Many Model

- The many-to-many model multiplexes any number of user threads onto an equal or smaller number of kernel threads.
- The following diagram shows the many-to-many threading model where 6 user level threads are multiplexing with 6 kernel level threads. In this model, developers can create as many user threads as necessary and the corresponding Kernel threads can run in parallel on a multiprocessor machine. This model provides the best accuracy on concurrency and when a thread performs a blocking system call, the kernel can schedule another thread for execution.



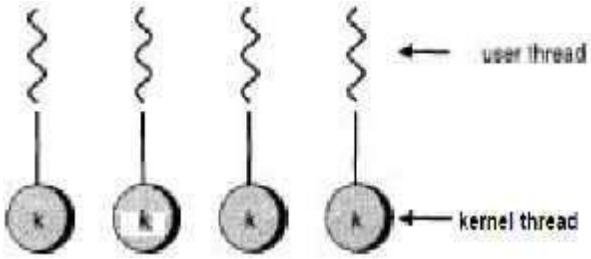
Many to One Model

- Many-to-one model maps many user level threads to one Kernel-level thread. Thread management is done in user space by the thread library. When thread makes a blocking system call, the entire process will be blocked. Only one thread can access the Kernel at a time, so multiple threads are unable to run in parallel on multiprocessors.
- If the user-level thread libraries are implemented in the operating system in such a way that the system does not support them, then the Kernel threads use the many-to-one relationship modes.



One to One Model

- There is one-to-one relationship of user-level thread to the kernel-level thread. This model provides more concurrency than the many-to-one model. It also allows another thread to run when a thread makes a blocking system call. It supports multiple threads to execute in parallel on microprocessors.
- Disadvantage of this model is that creating user thread requires the corresponding Kernel thread. OS/2, windows NT and windows 2000 use one to one relationship model.



(c)Thread libraries provide programmers with an API for creating and managing threads.

Thread libraries may be implemented either in user space or in kernel space. The former involves API functions implemented solely within user space, with no kernel support. The latter involves system calls, and requires a kernel with thread library support.

There are three main thread libraries in use today:

POSIX Pthreads - may be provided as either a user or kernel library, as an extension to the POSIX standard.

Win32 threads - provided as a kernel-level library on Windows systems.

Java threads - Since Java generally runs on a Java Virtual Machine, the implementation of threads is based upon whatever OS and hardware the JVM is running on, i.e. either Pthreads or Win32 threads depending on the system

22. Consider the following set of processes with the length of the CPU burst given in milliseconds (11) (NOV'16)

Process	Burst time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

The processes arrived in the order P1,P2,P3,P4,P5 at time 0.

(a) Draw Gantt charts that illustrate the execution of these processes using the following svheduling algorithms : FCFS,SJF ,non-preemptive priority (a smaller priority number implies higher priority)

(b) What is the turnaround time of all processes for each scheduling algorithms.

FCFS:

Gantt chart:

P1	P2	P3	P4	P5

0 10 11 13 14 19

Waiting time

Process	waiting time
P1	0
P2	10
P3	11
P4	13
P5	14

Average waiting time=(0+10+11+13+14)/5 = 9.6ms

Turn around time (TAT):

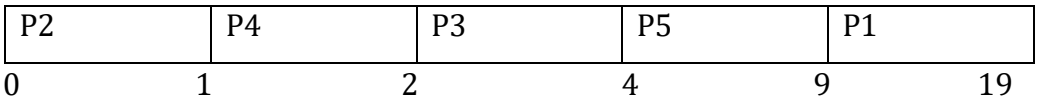
TAT=waiting time + burst time

Process	TAT
P1	10
P2	11
P3	13
P4	14
P5	19

Average TAT=(10+11+13+14+19)/5=13.4 ms

SJF:

Gantt chart:



Waiting time

Process	waiting time
P1	9
P2	0
P3	2
P4	1
P5	4

Average waiting time=(9+0+2+1+4)/5 = 3.2ms

Turn around time (TAT):

TAT=waiting time + burst time

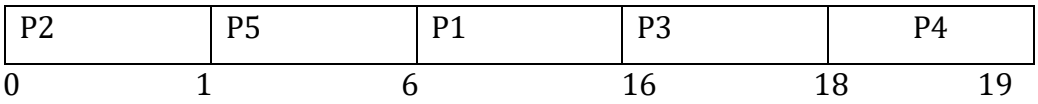
Process	TAT
P1	19
P2	1
P3	4
P4	2

P5	9

Average TAT=(19+1+4+2+9)/5=7 ms

NON-PREEMPTIVE PRIORITY SCHEDULING:

Gantt chart:



Waiting time

Process	waiting time
P1	6
P2	0
P3	16
P4	18
P5	1

Average waiting time=(6+0+16+18+1)/5 = 8.2ms

Turn around time (TAT):

TAT=waiting time + burst time

Process	TAT
P1	16
P2	1
P3	18
P4	19
P5	6

Average TAT=(16+1+18+19+6)/5=12 ms

23. Write short notes on Semaphores(APR'16) (11)

- It is a Synchronization tool that does not require busy waiting.
- A Semaphore *S* is an integer variable can only be accessed via two indivisible (atomic) operations wait and signal.

Wait (*S*):

while *S*≤ 0 do *no-op*;

S--;

signal (*S*):

S++;

Types of semaphore:

There are two types of semaphores:

1. Binary Semaphores

2. Counting semaphores

Binary Semaphore:

A binary semaphore is a semaphore with an integer value that can range only between 0 and 1. **A** binary semaphore can be simpler to implement than a counting semaphore, depending on the underlying hardware architecture.

- There are 2 processes p1 and p2 .p1 checks the valur of S,initially S=1 therefore s!=0. Since S- - makes s=0 so p1 enter to its CS .At the same time p2 also want to execute CS ,it checks S value which is already 0 so that it understands some process is in CS.so that p2 has to wait.
- After p1 finishes,it increments S by 1 therefore now S=1.now p2 checks S value,which is not equal to 0 ,so that p2 will decrement S by 1 therefore S=0 no p2 enter to its CS.

Counting semaphore:

Counting semaphore can be implemented using binary semaphores.Let S be a counting semaphore. To implement it in terms of binary semaphores we need the following data structures:

binary-semaphore S1, S2;

int C;

Initially S1 = 1, S2 = 0, and the value of integer C is set to the initial value of the counting semaphore S.

The wait operation on the counting semaphore S can be implemented as

follows:

```
wait (S1) ;
c--;
i f (C < 0)
{
signal(S1) ;
wait (S2) ;
}
signal(S1);
```

The signal operation on the counting semaphore S can be implemented as follows:

```
w a i t (S1) ;
C++ ;
i f (C <= 0)
signal (S2) ;
e l s e
signal (S1) ;
```

Example:

- A common paradigm for cooperating processes, A producer process produces information that is consumed by a consumer process.
- Have a buffer of items filled by the producer and emptied by the consumer
  - unbounded-buffer no limit on the size of the buffer.
  - bounded-buffer assumes that there is a fixed buffer size.
- Buffer is a shared memory .Producer and consumer run concurrently and must be synchronized. In bounded buffer, the consumer must wait if the buffer is empty and the producer must wait if the buffer is full.

In Unbounded Buffer,the consumer may have to wait for new items,but the producer can always produce new items

Bounded-Buffer Problem

- Shared data

  semaphore full, empty, mutex;

  Initially:

  full = 0, empty = n, mutex = 1

  Bounded-Buffer Problem Producer Process

```
do {
        …
    produce an item in nextp
        …
    wait(empty);
    wait(mutex);
        …
    add nextp to buffer
        …
    signal(mutex);
    signal(full);
} while (1);
```

- Count =number of buffers
- The producer produces an item and access buffer by decrementing count by 1 and sets mutex=0 and then adds item to buffer. after that producer leaves CS by setting mutex=1 ,increments full buffers by 1.

  Bounded-Buffer Problem Consumer Process

```
do {
    wait(full)
    wait(mutex);
        …
    remove an item from buffer to nextc
        …
```

```
                signal(mutex);
                signal(empty);
                        …
                consume the item in nextc
                        …
        } while (1);
```

- The consumer decrementing full buffer by 1 and then enters CS by setting mutex=0 after that consumes an item ,while leaving CS sets mutex=1 and increments empty buffers by 1.

24. Calculate average waiting time and average turnaround time for the following algorithms'

(a)FCFS

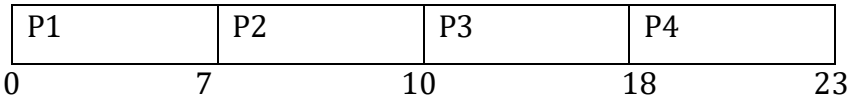
(b) Preemptive SJF(SRTF)

(c) Round robin(quantum = 1ms)

Process	Arrival time	Burst time
P1	0	7
P2	1	3
P3	2	8
P4	3	5

FCFS:

Gantt chart:



Waiting time

Process	waiting time
P1	0-0=0
P2	7-1=6
P3	10-2=8
P4	18-3=15

Average waiting time=(0+6+8+15)/4 = 7.25ms

Turn around time (TAT):

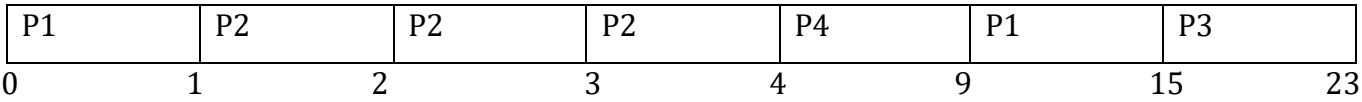
TAT=waiting time + burst time

Process	TAT
P1	7
P2	10
P3	18
P4	23

Average TAT=(7+10+18+23)/4=14.5 ms

Preemptive SJF(SRTF):

Gantt chart:



Waiting time

Process	waiting time
P1	9-0-1=8
P2	3-1-2=0
P3	15-2=13
P4	4-3=1

Average waiting time=(8+0+13+1)/4 = 5.5ms

Turn around time (TAT):

TAT=waiting time + burst time

Process	TAT
P1	15
P2	4
P3	23
P4	9

Average TAT=(15+4+23+9)/4=12.75 ms

Round robin (quantum = 1ms**):**

Gantt chart:

P1	P2	P3	P4	P1	P2	P3	P4	P1	P2	P3	P4	P1	P3	P4	P1	P3	P4	P1	P3	P1	P3	P3

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23

Waiting time

Process	waiting time
P1	20-6=14
P2	9-2=7
P3	22-7=15
P4	17-4=13

Average waiting time=(14+7+15+13)/4 = 12.25ms

Turn around time (TAT):

TAT=waiting time + burst time

Process	TAT
P1	21
P2	10
P3	23
P4	18

Average TAT=(21+10+23+18)/4=18 ms

25. (a)List the functions performed by dispatcher (2) (**APR'17**)

(b) Consider the following set of processes with the length of the CPU burst given in milliseconds

Process	Burst time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

The processes arrived in the order P1,P2,P3,P4,P5 at time 0.

(a) Draw Gantt charts that illustrate the execution of these processes using the following svheduling algorithms : FCFS,SJF ,non-preemptive priority (a smaller priority number implies higher priority) and RR (quantum=1) (3)

(b) What is the turnaround time of all processes for each scheduling algorithms in (i) (2).

(iii)What is the waiting time of each process for each of the scheduling algorithms in (i) (2)

(iv)Which of the algorithms in part (i) results in the minimum average waiting time (over all processes)? (2)

(Refer Q.no : 22 for FCFS,SJF,non-preemptive priority)

Round robin (quantum = 1ms**):**

Gantt chart:

P1	P2	P3	P4	P5	P1	P3	P5	P1	P5	P1	P5	P1	P5	P1	P1	P1	P1	P1

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19

Waiting time

Process	waiting time
P1	14-5=9
P2	1
P3	6-1=5
P4	3
P5	13-4=9

Average waiting time=(9+1+5+3+9)/5 = 5.4ms

Turn around time (TAT):

TAT=waiting time + burst time

Process	TAT
P1	19
P2	2
P3	7
P4	4
P5	14

Average TAT=(21+10+23+18)/4=21.2 ms

Conclusion SJF results in the minimum average waiting time .

Pondicherry University Questions

2 Marks

1. What is a Dispatcher? (UQ NOV '11) (Ref.Pg.No.2 Qn.No.9)
2. Define throughput? (UQ NOV '11 &APR'14) (Ref.Pg.No.3 Qn.No.12)
3. Define race condition? (UQ APR'14) (Ref.Pg.No.3 Qn.No.14)
4. What is critical section problem? (UQ APR'11, NOV'18) (Ref.Pg.No.3 Qn.No.15)
5. Define Aging and starvation? (APR' 14) (Ref.Pg.No.7 Qn.No.28)
6. What is context switch?(APR '11) (Ref.Pg.No.7 Qn.No.29)
7. What are the benefits of multithreaded programming? (NOV'14)) (Ref.Pg.No1 Qn.No.2)
8. What are the requirements that a solution to the critical section problem must satisfy? (NOV'14)) (Ref.Pg.No.4 Qn.No.16)
9. What is a thread? (APR'15, NOV '15) (Ref.Pg.No.1 Qn.No.1)
10. Define CPU scheduling (APR'15)) (Ref.Pg.No.2 Qn.No.7)
11. What is a monitor? (NOV '15) (Ref.Pg.No.7 Qn.No.30)
12. Define Medium Term Scheduler (NOV'16)(Ref.Pg.No.8 Qn.No.32)
13. What is Critical Regions? List down various mechanisms used to deal with critical region problem.(NOV'16)(Ref.Pg.No.8 Qn.No.33)
14. What is the difference between process and thread? (May 2017 Ref.Pg.No.6 Qn.No.24)
15. What are the uses of job queues, ready queue and device queue? (MAY'17)(Ref.Pg.No.7 Qn.No.31)
16. Compare user threads and kernel threads? (May'16) (Ref.Pg.No.1 Qn.No.4)
17. What are the requirements that a solution to the critical section problem must satisfy? (NOV'14) (may'16) (Ref.Pg.No.4 Qn.No.16)
18. Write about real time scheduling? (NOV 18) (Ref.Pg.No.34 Qn.No.52)

11 MARKS

1. Write short notes on CPU scheduler? (UQ NOV '13, APR'11, NOV '15) (Ref.Pg.No.9 Qn.No.1)
2. Explain FCFS? (UQ NOV '13) (Ref.Pg.No.10 Qn.No.2)
3. Explain Multi level queue scheduling?(UQ NOV '11) (Ref.Pg.No.15 Qn.No.6)
5. Write in detail about Critical section problem? (UQ NOV '11) (Ref.Pg.No.17 Qn.No.9)
6. Write short notes on Multiple-Processor Scheduling and real time scheduling?(UQ NOV '11 & APR'12) (Ref.Pg.No.16 Qn.No.8)

7. Write short notes on Synchronization Hardware? (UQ NOV '11 &NOV '13) (Ref.Pg.No.20 Qn.No.11)

8. Write short notes on Semaphores?(UQ NOV '12 , NOV '13, NOV '15) (Ref.Pg.No.22 Qn.No.12)

9. What is critical section problem and explain two process solution and multiple process solutions? (APR'15) (Ref.Pg.No.15 Qn.No.9)

10. Explain producer-consumer problem? (UQ APR'14) (Ref.Pg.No.26 Qn.No.15)

11. Explain in detail about the threading issues (APR'15)) (Ref.Pg.No.35 Qn.No.19)

12. Write short notes on Dining-Philosophers Problem (NOV'14) (Ref.Pg.No.24 Qn.No.13)

13. Write short notes on Readers-Writers Problem? (UQ APR'14,NOV'14) (Ref.Pg.No.25 Qn.No.14)

14. Explain Bakery Algorithm OR Multiple process solution (APR'15)) (Ref.Pg.No.19 Qn.No.10)

15. Describe the difference between wait(A) where A is a semaphore and B Wait() ,where B is a condition variable in a monitor. (APR'15) (Ref.Pg.No.37 Qn.No.21)

16. Explain briefly about round robin scheduling with diagram. (NOV '15) (Ref.Pg.No.14 Qn.No.5)

17. (a)List the functions performed by dispatcher (2) (**APR'17**) (Ref.Pg.No2. Qn.No.9)

(b)Consider the following set of processes with the length of the CPU burst given in milliseconds (**APR'17**) (Ref.Pg.No52. Qn.No.34)

Process	Burst time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

The processes arrived in the order P1,P2,P3,P4,P5 at time 0.

(c) Draw Gantt charts that illustrate the execution of these processes using the following svheduling algorithms : FCFS,SJF ,non-preemptive priority (a smaller priority number implies higher priority) and RR (quantum=1) (3)

(d) What is the turnaround time of all processes for each scheduling algorithms in (i) (2).

(iii)What is the waiting time of each process for each of the scheduling algorithms in (i) (2)

(iv)Which of the algorithms in part (i) results in the minimum average waiting time (over all processes)? (2)

18. Calculate average waiting time and average turnaround time for the following algorithms . (Ref.Pg.No48. Qn.No.24)

(a) FCFS

(b) Preemptive SJF(SRTF)

(c) Round robin(quantum = 1ms)

Process	Arrival time	Burst time
P1	0	7
P2	1	3
P3	2	8
P4	3	5

19. Write short notes on Semaphores and its types with an example (APR'16, NOV '18) (11)

(Ref.Pg.No44. Qn.No.23)

20. Consider the following set of processes with the length of the CPU burst given in milliseconds (11) (NOV'16, NOV '18)

Process	Burst time	Priority
P1	10	3
P2	1	1
P3	2	3
P4	1	4
P5	5	2

The processes arrived in the order P1,P2,P3,P4,P5 at time 0.

(c) Draw Gantt charts that illustrate the execution of these processes using the following svheduling algorithms : FCFS,SJF ,non-preemptive priority (a smaller priority number implies higher priority)

(d) What is the turnaround time of all processes for each scheduling algorithms.

(Ref.Pg.No.42 Qn.No.22)

21. (a)List the requirements that must be satisfied by critical section problem (4 marks)

(b)Discuss the relationship between user threads and kernel threads (5 marks)

(c) List the names of thread semaphore

(2 marks) (APR'17)(Ref.Pg.No.40 Qn.No.21)

22. Write short notes on Monitors (OR) Explain the structure of monitors . Give the solution for Dining-philosopher problem using monitor (11) (NOV'16, NOV '18)

(Ref.Pg.No.31 Qn.No.17)

23. Consider the following set of processes with the length of the CPU burst given in milliseconds (11) (NOV '18)

Process	Burst time
P1	6
P2	10
P3	3
P4	4
P5	2

a) Draw the Gantt charts illustrating execution of these processes for round robin scheduling (quantum=2)& FCFS

b) Calculate waiting time for each process for each scheduling algorithm.

c) Calculate average waiting time for each scheduling algorithm.

Consider all processes arrive in order P1,P2,P3,P4,P5 at time zero.